

Events by MC

Visão geral da arquitetura

Orientação a objetos, arquitetura hexagonal, APIs REST, Vue 3 e persistência

STACK	ESTADO DO PROJETO
Java 21 + HttpServer + JDBC + PostgreSQL	Aplicação web em desenvolvimento; backend sem Spring Boot

Resumo executivo

O sistema reúne catálogo de eventos, programação, inscrições, agenda, presença por atividade, avaliações, relatórios, certificados, interação e moderação administrativa. O frontend é uma SPA Vue 3 com TypeScript; o backend oferece rotas HTTP/JSON em Java 21; o PostgreSQL mantém dados e restrições de integridade. A separação em domínio, casos de uso, portas e adaptadores mostra a proposta hexagonal, com algumas regras ainda concentradas no JDBC por dependerem de transações e concorrência.

Nota de escopo: este documento descreve o código inspecionado, não comprova a operação em produção. Nenhum arquivo-fonte foi modificado nem testes foram executados para produzir este PDF.

O mapa em uma frase

A tela pede uma operação à API; o handler traduz HTTP; o caso de uso aplica a política da operação; o domínio protege invariantes; uma porta permite que o adaptador acesse banco ou serviço externo - e a resposta percorre o caminho de volta.

1. Fluxo geral e arquitetura hexagonal

A arquitetura hexagonal separa o núcleo de negócio dos mecanismos de entrada e saída. HTTP e PostgreSQL são detalhes externos: importantes para executar a aplicação, mas não devem definir as regras do domínio. A ligação concreta acontece no CompositionRoot.

1 Navegador e Vue

Telas, componentes, rotas SPA e estado da sessão



2 HTTP e proxy

fetch + JSON + Bearer JWT; /api via Vite ou Nginx



3 Entrada Java

Filtro, Router e handlers convertem HTTP em operações



4 Aplicação e domínio

Portas de entrada, casos de uso, entidades e regras



5 Saída e recursos externos

Portas de saída -> JDBC, JWT, BCrypt e SMTP -> PostgreSQL

Direção das dependências

Os handlers dependem de interfaces de entrada; os casos de uso dependem de interfaces de saída; os adaptadores JDBC/JWT implementam essas interfaces. Assim, é possível testar regras com repositórios em memória e trocar tecnologia externa sem reescrever a operação inteira.

CAMADA	RESPONSABILIDADE	EXEMPLOS
Domínio	Estados, entidades e invariantes do negócio	Evento, Usuario, Atividade
Aplicação	Orquestra operações e permissões	EventosUseCase, PresencaUseCase
Portas	Contratos de entrada e saída	OperacoesPresenca, PresencaRepository
Adaptadores	Convertem tecnologia em contratos	PresencaHandler, JdbcPresencaRepository
Montagem	Escolhe as implementações concretas	CompositionRoot

2. Orientação a objetos no código

Encapsulamento e invariantes

Evento guarda seu estado e oferece operações como publicar(), encerrar(), suspender() e excluir(). Publicar exige período e local; encerrar exige publicação prévia. Em vez de expor um setter genérico, o objeto nomeia as transições permitidas. Usuario controla seu conjunto de perfis: uma conta não aceita VISITANTE e não pode ficar sem perfil autenticado.

Composição, abstração e polimorfismo

Evento contém uma Programacao, que organiza atividades e rejeita IDs duplicados. O polimorfismo principal não vem de herança entre entidades: vem de interfaces. PresencaUseCase usa PresencaRepository; em produção recebe JDBC e, em testes, pode receber uma implementação em memória. Esse é um exemplo concreto de inversão de dependência.

CONCEITO DE POO	EXEMPLO NO PROJETO	POR QUE IMPORTA
Entidade	Evento e Usuario com identidade UUID	Mantêm estado e comportamento relacionados
Objeto de valor simples	Pessoa, Senha e regras de chamada	Concentram validações e significado
Interface	EventoRepository, TokenProvider	Desacoplam o caso de uso da tecnologia
Composição	Evento possui Programacao	Expressa relação entre objetos sem herança
Record/DTO	DadosEvento e LoginRespostaDTO	Transportam dados sem expor entidades inteiras

A divisão de responsabilidades

- **Entidade:** 'esta transição é válida para o evento?'
- **Caso de uso:** 'quem pediu, tem permissão e qual operação executar?'
- **Repositório:** 'como consultar ou gravar com segurança e atomicidade?'
- **Handler:** 'como transformar HTTP/JSON em chamada Java e resposta HTTP?'

A implementação é hexagonal de forma pragmática: parte das regras de inscrição, vagas, agenda e presença está no adaptador JDBC porque requer bloqueios, unicidade e transações. Isso merece atenção didática, mas não invalida a separação geral.

3. APIs REST: do clique à resposta

Uma operação da API é definida por método HTTP, caminho, dados opcionais e credencial. O navegador usa /api; Vite ou Nginx remove esse prefixo e encaminha a solicitação ao Java. Assim, POST /api/eventos no navegador chega à rota POST /eventos do Router.

Exemplo: criar evento

- A EventFormView coleta dados e valida antes do envio.
- eventoService usa apiRequest para enviar JSON via POST /api/eventos.
- httpClient inclui Authorization: Bearer <JWT> nas operações protegidas.
- O filtro valida o token; Router chama GestaoEventosHandler.
- EventosUseCase verifica organizador, campos e estado; Evento protege suas invariantes.
- JdbcEventoRepository salva no PostgreSQL; o handler devolve JSON e HTTP 201.

Principais grupos de rotas

ÁREA	EXEMPLOS COM PREFIXO DO WEBAPP
Conta	POST /api/usuarios; POST /api/auth/login; GET /api/usuarios/me
Eventos	GET /api/eventos; GET /api/eventos/{id}; POST /api/eventos
Programação	GET /api/eventos/{id}/programacao; GET/POST /api/eventos/{id}/trilhas
Participação	POST /api/eventos/{id}/inscricoes; GET /api/usuarios/me/agenda
Presença	POST /api/atividades/{id}/chamadas; POST /api/presencas/confirmacoes
Avaliações	GET /api/eventos/{id}/questionario; POST /api/eventos/{id}/avaliacoes
Relatórios	GET /api/eventos/{id}/relatorios/frequencia(.csv ou .pdf)
Administração	GET /api/admin/eventos; DELETE /api/admin/eventos/{id}

Segurança: o frontend pode ocultar telas, mas quem autoriza é o backend. JWT prova a identidade, perfis limitam ações e os casos de uso conferem propriedade do evento ou inscrição. Erros frequentes: 400 (dados), 401 (sem sessão), 403 (sem permissão), 404 (não encontrado), 409 (conflito) e 429 (tentativas em excesso).

4. Frontend Vue 3 e o princípio de SPA

main.ts instala Vue, Pinia e Router. App.vue mantém a estrutura comum e usa RouterView para trocar a tela sem recarregar o HTML inteiro. O Router usa histórico do navegador, carregamento sob demanda e guardas de navegação para participante, organizador e administrador.

PASTA	PAPEL
views/	Telas completas: home, detalhes, login, painéis e administração
features/	Serviços, tipos, utilitários e componentes por funcionalidade
components/	Layout, cards, ícones e elementos reaproveitáveis
shared/services/	Cliente HTTP centralizado: fetch, JWT, erros e downloads
stores/	Estado compartilhado da sessão com Pinia
router/	Caminhos, títulos, carregamento de páginas e guardas

Conceitos Vue no exemplo da home

- ref guarda lista de eventos, mensagem de erro e estado de carregamento.
- computed deriva busca, categoria, locais e lista visível; não duplica estado.
- onMounted consulta a API quando a tela aparece.
- Componentes como EventFilters e EventGrid separam interação e apresentação.
- A função filtrarEventos combina termo, categoria e local; normalizarTexto remove acentos.

Fronteira entre tela e API

A view não deve espalhar fetch por todo o projeto. Ela chama um serviço TypeScript; o serviço conhece o contrato REST; httpClient aplica o comportamento comum. Os tipos de interface descrevem os dados esperados, enquanto o mapeamento no serviço adapta nomes e valores para uso visual. A store guarda token e perfis na sessionStorage e encerra a sessão em 401.

Atenção: permissões do Router são experiência de navegação, não barreira de segurança. Também há tipos e prévia de banners/fotos, mas ainda não há upload nem persistência de mídia no backend.

5. Presença, PostgreSQL e infraestrutura

Fluxo de QR e código

A chamada é vinculada a uma atividade. O organizador gera um código aleatório de 12 caracteres; a API guarda apenas o hash SHA-256 e a validade de cinco minutos. O QR abre a tela, mas não marca presença automaticamente. O participante deve entrar na conta, estar inscrito e confirmar o código. Nova chamada revoga a anterior, e o banco impede duplicidade por usuário/atividade.

Banco de dados

Os scripts V1 a V10 evoluem o esquema: usuários, perfis, eventos, atividades, inscrições, presenças, programação, agenda, questionários, mensagens, certificados, políticas de frequência e moderação. Chaves estrangeiras e restrições CHECK/UNIQUE protegem integridade mesmo quando duas requisições chegam juntas. JDBC usa consultas parametrizadas e transações nas operações sensíveis.

Execução

SERVIÇO	FUNÇÃO
PostgreSQL	Armazena dados; volume Docker preserva o banco
API Java	Servidor HttpServer; autenticação, regras e acesso JDBC
Nginx + Vue	Entrega a SPA e encaminha /api à API na rede interna

A API lê DB_URL, DB_USERNAME, DB_PASSWORD e JWT_SECRET do ambiente. SMTP é opcional para envio de certificados. No Docker, scripts do PostgreSQL executam automaticamente somente quando o volume está vazio. Banco existente requer aplicação controlada das versões pendentes após backup.

Testes existentes

Há JUnit para domínio e casos de uso, testes HTTP com servidor e JWT reais usando repositórios falsos, testes JDBC que exigem banco configurado, Vitest para serviços e utilitários Vue e testes do esquema SQL. Esta geração de PDF foi somente documental: não houve nova execução de testes.

6. Diagnóstico e roteiro de estudo

O que já está bem estruturado

- Entidades com operações nomeadas e proteção de estados importantes.
- Portas e injeção de dependências que permitem testes sem PostgreSQL.
- Cliente HTTP único no frontend e serviços separados por funcionalidade.
- Autenticação no backend, checagem de propriedade e restrições no banco.
- Migrações versionadas e deploy local com três serviços.

Pontos que ainda pedem evolução

- Completar edição de conta, eventos e atividades, além de conflitos de local/horário.
- Permitir correção auditável da presença manual e amadurecer moderação de mensagens.
- Implementar armazenamento e entrega de banners/fotos se permanecerem no escopo.
- Validar ponta a ponta com PostgreSQL real, SMTP e leitura de QR no celular.
- Rever regras hoje embutidas no JDBC e documentar melhor padrões de POO2.

Ordem recomendada para aprender com o próprio código

1. Siga GET /eventos: a consulta pública mais simples. 2. Siga POST /eventos: validação, autorização e persistência. 3. Estude login: BCrypt, JWT, filtro e sessão Pinia. 4. Estude presença: QR, hash, inscrição, transação e UNIQUE. Para cada fluxo, pergunte: quem recebe, quem decide, qual objeto protege a regra e qual adaptador conversa com o exterior?

Mapa de arquivos para consulta

Caminhos abaixo são relativos à raiz do repositório events-api.

TEMA	ARQUIVO
Inicialização	src/main/java/br/com/eventsbymc/eventsapi/EventsApiApplication.java
Montagem	src/main/java/br/com/eventsbymc/eventsapi/infrastructure/config/CompositionRoot.java
Domínio	src/main/java/br/com/eventsbymc/eventsapi/domain/model/Evento.java
Presença	src/main/java/br/com/eventsbymc/eventsapi/application/usecase/PresencaUseCase.java
Rotas	src/main/java/br/com/eventsbymc/eventsapi/infrastructure/adapters/in/web/RotasApi.java
Vue/HTTP	frontend/src/router/index.ts; frontend/src/shared/services/httpClient.ts
Banco	src/main/resources/db/migration/; db/README.md